# Algoritmos y Estructuras de Datos II

Laboratorio - 14/05/2026

# Laboratorio 4: Memoria dinámica y punteros. Introducción a TADs.

- Revisión 2026: Franco Luque / Marco Rocchietti

## Recursos

Recursos generales:

- Videos del Laboratorio en el aula virtual
- Documentación en el aula virtual
  - **La biblia: El lenguaje de programación C.**
- Estilo de codificación:
  - Guía de estilo para la programación en C
  - Consejos de Estilo de Programación en C
- Más:
  - Referencia de C (cppreference)
  - The GNU C Library Reference Manual (en inglés)
  - Páginas del manual de linux: Comando “man 3 <*función*>”. Ejemplo: “man 3 scanf”.

Recursos específicos:

- Teóricos:
  - Tipos Concretos
  - TADs
- Prácticos:
  - Práctico 2.1
  - Práctico 2.2
- **Punteros: Comparación del teórico/práctico y el laboratorio**
- Punteros: tipos * , desreferenciación o indirección (*), referenciación o dirección (&)
  - 04 - Punteros en C
- Memoria dinámica: sizeof(), malloc(), y free() (también malloc() y realloc() )
  - 06 - Memoria Dinámica en C
- Visualizador para C: https://pythontutor.com/c.html#mode=edit
- Violación de segmento

# Primera parte: Memoria dinámica

## **Ejercicio 1:** Gomería de motos

En este ejercicio se define una estructura de datos para representar una moto. Una moto tiene **dos ruedas**, y cada rueda tiene un número que indica la **presión**, y un booleano que indica si está **pinchada** o no. Una moto en condiciones correctas tiene ambas ruedas con **presión óptima** y sin pinchar.

En `moto.h` se define el tipo `moto_t` y se declaran algunas funciones que se implementan en `moto.c`. El tipo `moto_t` se define usando **punteros y estructuras**. En el siguiente diagrama se puede ver un ejemplo de una moto nueva:



**Ejercicio 1.1:** Implementar la función `reparar_moto()`, especificada de la siguiente manera:

```
/**
 * @brief Repara las ruedas de la moto
 *
 * Si una rueda no está pinchada, se le debe poner la presión óptima
 * SIN CAMBIARLA.
 * Si una rueda está pinchada, se debe:
 *   1. Descartar la rueda pinchada (liberar memoria)
 *   2. Colocar una rueda nueva con la presión correcta (alojar memoria)
 */
void reparar_moto(moto_t moto);
```

Compilar y testear con:

```
$ gcc -Wall -Wextra -std=c99 tests.c moto.c -o tests
$ ./tests
```

Se incluye un **ejemplo** simple de uso en `ejemplo.c`. Compilar y ejecutar con:

```
$ gcc -Wall -Wextra -std=c99 ejemplo.c moto.c -o ejemplo
$ ./ejemplo
```

# **Ejercicio 2:** El brazo robótico

En este ejercicio se define una estructura de datos para representar un **brazo robótico.** El brazo tiene **un codo** y **una mano que sale del codo**. Cada parte tiene una cantidad de aceite y un booleano que indica si está **en cortocircuito** o no. Un robot en condiciones correctas tiene sus partes con **aceite óptimo** y sin cortocircuitos.

En `robot.h` se define el tipo `robot_t` y se declaran algunas funciones que se implementan en `robot.c`. El tipo `robot_t` se define usando **punteros y estructuras**. En el siguiente diagrama se puede ver un ejemplo de **robot nuevo en condiciones correctas**:



**IMPORTANTE: Observar que la mano se desprende del codo.**

**Ejercicio 2.1:** Implementar la función `reparar_robot()`, especificada de la siguiente manera:

```
/**
 * @brief Repara el robot
 *
 * Reemplaza SOLAMENTE las partes quemadas por partes nuevas con aceite óptimo
 * (requiere liberar memoria y alojar memoria nueva)
 *
 * A las partes no quemadas NO LAS REEMPLAZA, sólo les pone el aceite óptimo
 */
```

Compilar y testear con:

```
$ gcc -Wall -Wextra -std=c99 tests.c robot.c -o tests
$ ./tests
```

Se incluye un **ejemplo** simple de uso en `ejemplo.c`. Compilar y ejecutar con:

```
$ gcc -Wall -Wextra -std=c99 ejemplo.c robot.c -o ejemplo
$ ./ejemplo
```

# Segunda parte: Principios de listas enlazadas

## **Ejercicio 3:** Construcción de listas enlazadas

Se proveen en **`main.c`** definiciones de tipo para listas enlazadas y sus nodos:

```
typedef struct Node {
   int data;
   struct Node* next;
} Node;

typedef Node *List;
```

Se debe **completar la implementación de la función `setup_example()`** que construye y devuelve una lista de ejemplo de 3 elementos: [ 20, 10, 0 ] . La lista debe construirse de la misma forma que en el siguiente pseudo-código:

```
type node = tuple
    data: int
    next: pointer of node
end tuple
type list = pointer of node

var i: int
var my_list: list
var a_node: pointer of node

i := 3

alloc(my_list)
a_node := my_list
i := i - 1 // first node built

while i > 0 do
    a_node->data := i*10
    alloc(a_node->next)
    a_node := a_node->next
    i := i - 1
od

a_node->data := 0
a_node->next := null
a_node := null
```

Compilar, ejecutar y verificar que la salida en pantalla es la correcta:

```
$ gcc -Wall -Wextra -pedantic -std=c99 main.c -o main
$ ./main
[ 20, 10, 0, ]
```

# **Ejercicio 3-b:** Visualización de listas enlazadas en C

Tomar el código del ejercicio anterior e introducirlo en el siguiente sitio que permite la visualización de ejecuciones de código en lenguaje C:

https://pythontutor.com/c.html#mode=edit

1. **Entrar a visualizar la ejecución haciendo click en:**



2. **Realizar la ejecución paso a paso haciendo click en “Next”:**



3. **Responder:**

- **¿Cómo funciona el algoritmo de construcción de listas?**
- **¿Qué sucede con otros valores iniciales de “i”? Probar 4, 5, 6…**

**Ayuda:** El estado final debería verse de la siguiente manera:

## **Ejercicio 4:** Agregando un elemento al final

Además de `setup_example()` del ejercicio anterior, completar en `main.c` la implementación de la función `append_example()` que agrega un elemento de ejemplo al final de la lista. Basarse en el siguiente pseudo-código:

```
var a_node: pointer of node

a_node := my_list

while a_node->next != null do
    a_node := a_node->next
od

alloc(a_node->next)
a_node->next->data := 88
a_node->next->next := null
```

1. **Compilar, ejecutar y verificar que la salida en pantalla es la correcta:**

```
$ gcc -Wall -Wextra -pedantic -std=c99 main.c -o main
```

```
$ ./main
Lista antes del append: [ 20, 10, 0, ]
Lista después del append: [ 20, 10, 0, 88, ]
```

2. **Visualizar en https://pythontutor.com/c.html#mode=edit**

## **Ejercicio 5:** Eliminando un elemento al principio

Completar en `main.c` la implementación de la función `tail_example()` que elimina el primer elemento de una lista. Basarse en el siguiente pseudo-código:

```
var a_node: pointer of node

a_node := my_list
my_list := my_list->next
free(a_node)
a_node := null
```

1. **Compilar, ejecutar y verificar que la salida en pantalla es la correcta:**

```
$ gcc -Wall -Wextra -pedantic -std=c99 main.c -o main
$ ./main
Lista antes del tail: [ 20, 10, 0, ]
Lista después del tail: [ 10, 0, ]
```

2. **Visualizar en https://pythontutor.com/c.html#mode=edit**

# Tercera parte: Primeros TADs

## Preliminares: TADS

### Encapsulamiento: Separación de especificación e implementación

Lo primero que debemos observar es la forma en la que logramos mantener separadas la especificación del TAD de su implementación. Cuando definimos un TAD es deseable garantizar **encapsulamiento**, es decir, que solamente se pueda acceder y/o modificar su estado a través de las operaciones provistas.

No todos los lenguajes brindan las mismas herramientas para lograr una implementación *opaca* y se debe usar el mecanismo apropiado según sea el caso. Particularmente el lenguaje del teórico-práctico separa la especificación de un TAD de su implementación utilizando las signaturas **`spec` … `where`** e **`implement` … `where`** respectivamente. En este laboratorio se debe buscar la manera de lograr encapsulamiento usando el lenguaje **C**.

## Precondiciones y postcondiciones

A diferencia del práctico, a las *precondiciones* y *postcondiciones* de los métodos **sí vamos a verificarlas** (en la medida de lo posible). Recordar que nuestros programas deben ser **robustos**, por lo tanto cuando corresponda usaremos `assert()` para garantizar el cumplimiento de las pre y post condiciones de los métodos. Esta práctica es propia de la etapa de desarrollo de un programa, y una vez que el mismo está finalizado, verificado y listo para desplegarlo en producción, se pueden eliminar las aserciones mediante un flag de compilación (-DNDEBUG).

# Ejercicio 6: TAD Par

Considerar la siguiente especificación del TAD Par

---

**spec** Pair **where**

**constructors**
  **fun** new(**in** x : int, **in** y : int) **ret** p : Pair
  *{- crea un par con componentes (x, y) -}*

**destroy**
  **proc** destroy(**in** p : Pair)
  *{- libera memoria en caso que sea necesario -}*

**operations**
  **fun** first(**in** p : Pair) **ret** x : int
  *{- devuelve el primer componente del par-}*

  **fun** second(**in** p : Pair) **ret** y : int
  *{- devuelve el segundo componente del par-}*

  **fun** swapped(**in** p : Pair) **ret** s : Pair
  *{- devuelve un nuevo par con los componentes de p intercambiados -}*

---

**a)** Abrir la carpeta `pair_a-tuple` y revisar la especificación del TAD en `pair.h`. Luego crear el archivo `pair.c` e implementar las funciones del TAD. Para probar la implementación usar el módulo `main.c` como programa de prueba.

**b)** Abrir la carpeta `pair_b-array` y revisar la nueva especificación del TAD en `pair.h`. Crear el archivo `pair.c` con la implementación de las funciones del TAD. Copiar el archivo `main.c` del apartado anterior al directorio `pair_b-array` y compilar.

I. ¿Por qué falla?
II. Hacer las modificaciones necesarias en `main.c` para que compile.
III. *¿El diseño del TAD logra encapsulamiento? ¿Por qué sí? ¿Por qué no?*

**c)** Abrir la carpeta `pair_c-pointer` y revisar la especificación del TAD en `pair.h`. Luego completar la implementación de las funciones y compilar usando el módulo `main.c` como programa de prueba. *¿La implementación logra encapsulamiento? ¿Por qué sí? ¿Por qué no?*

| | |
|---|---|
| ! | *Para definir constructores, destructores y operaciones de copia será necesario hacer manejo de **memoria dinámica** (pedir y liberar memoria en tiempo de ejecución). En este caso se necesita espacio suficiente para almacenar un valor de tipo* **`struct s_pair_t`**. |

**d)** Abrir la carpeta **`pair_d-pointer`** y revisar **`pair.h`**. Copiar el archivo **`pair.c`** del apartado anterior y agregar las definiciones necesarias para que compile con la nueva versión de **`pair.h`**. *¿La implementación logra encapsulamiento?* Copiar el archivo **`main.c`** del apartado anterior y compilar. *¿Está bien que falle?* Hacer las modificaciones necesarias en **`main.c`** para que compile sin errores.

**e)** Considerar la nueva especificación polimórfica para el TAD Pair:

```
spec Pair of A where

constructors
    fun new(in x : A, in y : A) ret p : Pair of A
    {- crea un par con componentes (x,  y) -}

destroy
   proc destroy(in p : Pair of A)
   {- libera memoria en caso que sea necesario -}

operations
    fun first(in p : Pair of A) ret x : A
    {- devuelve el primer componente del par-}

    fun second(in p : Pair of A) ret y : A
    {- devuelve el segundo componente del par-}

    fun swapped(in p : Pair of A) ret s : Pair of A
    {- devuelve un nuevo par con los componentes de p intercambiados -}
```

¿Qué diferencia hay entre la especificación anterior y la que se encuentra en el **`pair.h`** de la carpeta **`pair_d-pointer`**? Copiar **`pair.c`** del apartado anterior y modificarlo para utilizar la nueva interfaz especificada en **`pair.h`**. Pueden utilizar el **`main.c`** del apartado anterior para compilar.

## Ejercicio 7: TAD Contador

Dentro de la carpeta **`ej7`** se encuentran los siguientes archivos:

| *Archivo* | *Descripción* |
|---|---|
| **`counter.h`** | Contiene la especificación del TAD Contador. |
| **`counter.c`** | Contiene la implementación del TAD Contador. |
| **`main.c`** | Contiene al programa principal que lee uno a uno los caracteres de un archivo chequeando si los paréntesis están balanceados. |

**a)** Implementar el TAD Contador. Para ello deben abrir **`counter.c`** y programar cada uno de los constructores y operaciones cumpliendo la especificación dada en **`counter.h`**.

Recordar que deben verificar en **counter.c** todas las precondiciones especificadas en **counter.h** usando llamadas a la función **assert()**.

**b)** Usar el TAD Contador para chequear paréntesis balanceados. Para ello deben abrir el archivo **main.c** y entender qué es lo que hace la función **matching_parentheses()** y completar con llamadas al constructor y destructor del contador donde consideren necesario. ¡Es muy importante llamar al destructor del TAD una vez este no sea necesario para poder liberar el espacio de memoria que tiene asignado!

Una vez implementados los incisos **(a)**, **(b)** compilar ejecutando:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c counter.c main.c
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 counter.o main.o -o counter
```

Ahora se puede ejecutar el programa corriendo:

```
$ ./counter input/<file>.in
```

siendo **<file>** alguno de los nombres de archivo dentro de la carpeta **input**. Asegurarse que para aquellos archivos con paréntesis balanceados, al ejecutar el programa se imprima en pantalla

```
Parentheses match.
```

y para aquellos con paréntesis no balanceados imprima

```
Parentheses mismatch.
```